

EXPERIMENT 12

IMPLEMENTATION OF RSA ALGORITHM

AIM:

To implement the RSA (Rivest–Shamir–Adleman) algorithm in C for encryption and decryption of a message using public and private keys.

ALGORITHM:

Key Generation

1. Choose two prime numbers p and q .
2. Calculate:
$$n = p \times q$$
3. Calculate Euler's totient:
$$\phi(n) = (p - 1) \times (q - 1)$$
4. Choose an integer e such that:
$$1 < e < \phi(n) \text{ and } \gcd(e, \phi(n)) = 1.$$
5. Calculate d such that:
$$d \times e \equiv 1 \pmod{\phi(n)}.$$
6. The public key is (e, n) .
7. The private key is (d, n) .

Encryption

For plaintext M :

$$C = M^e \bmod n$$

where C is the ciphertext.

Decryption

For ciphertext C :

$$M = C^d \bmod n$$

where M is the original plaintext.

PROGRAM:

```
#include <stdio.h>
```

```
/* Function to find GCD */  
  
long long gcd(long long a, long long b)  
{  
    while (b != 0)  
    {  
        long long temp = b;  
        b = a % b;  
        a = temp;  
    }  
  
    return a;  
}
```

```
/* Function to calculate (base^exp) % mod */  
  
long long powerMod(long long base,  
                   long long exp,  
                   long long mod)  
{  
    long long result = 1;  
  
    base = base % mod;  
  
    while (exp > 0)  
    {  
        if (exp % 2 == 1)  
        {  
            result = (result * base) % mod;  
        }  
    }
```

```

        base = (base * base) % mod;
        exp = exp / 2;
    }

    return result;
}

/* Find modular inverse */
long long modInverse(long long e,
                    long long phi)
{
    long long d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
        {
            return d;
        }
    }

    return -1;
}

int main()
{
    long long p, q;
    long long n, phi;
    long long e, d;

```

```

long long message;
long long cipher;
long long decrypted;

printf("RSA ALGORITHM\n");
printf("-----\n");

printf("Enter prime number p: ");
scanf("%lld", &p);

printf("Enter prime number q: ");
scanf("%lld", &q);

/* Calculate n */
n = p * q;

/* Calculate Euler's Totient */
phi = (p - 1) * (q - 1);

/* Choose e */
printf("Enter value of e: ");
scanf("%lld", &e);

/* Check whether e is valid */
if (gcd(e, phi) != 1)
{
    printf("Invalid value of e.\n");
    printf("e must be relatively prime to phi(n).\n");
    return 1;
}

```

```

}

/* Calculate d */
d = modInverse(e, phi);

if (d == -1)
{
    printf("Modular inverse does not exist.\n");
    return 1;
}

printf("\nRSA Parameters\n");
printf("n = %lld\n", n);
printf("phi = %lld\n", phi);

printf("Public Key = (%lld, %lld)\n", e, n);
printf("Private Key = (%lld, %lld)\n", d, n);

/* Input message */
printf("\nEnter message (integer less than n): ");
scanf("%lld", &message);

if (message >= n)
{
    printf("Message must be smaller than n.\n");
    return 1;
}

/* Encryption */

```

```

cipher = powerMod(message, e, n);

/* Decryption */
decrypted = powerMod(cipher, d, n);

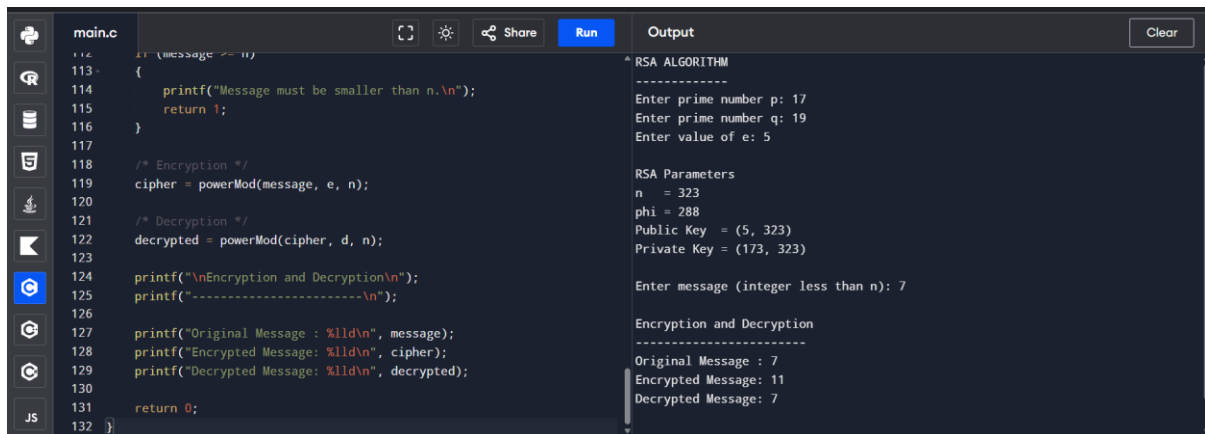
printf("\nEncryption and Decryption\n");
printf("-----\n");

printf("Original Message : %lld\n", message);
printf("Encrypted Message: %lld\n", cipher);
printf("Decrypted Message: %lld\n", decrypted);

return 0;
}

```

OUTPUT:



```

main.c
112  if (message >= n)
113  {
114      printf("Message must be smaller than n.\n");
115      return 1;
116  }
117
118  /* Encryption */
119  cipher = powerMod(message, e, n);
120
121  /* Decryption */
122  decrypted = powerMod(cipher, d, n);
123
124  printf("\nEncryption and Decryption\n");
125  printf("-----\n");
126
127  printf("Original Message : %lld\n", message);
128  printf("Encrypted Message: %lld\n", cipher);
129  printf("Decrypted Message: %lld\n", decrypted);
130
131  return 0;
132 }

```

```

RSA ALGORITHM
-----
Enter prime number p: 17
Enter prime number q: 19
Enter value of e: 5

RSA Parameters
n = 323
phi = 288
Public Key = (5, 323)
Private Key = (173, 323)

Enter message (integer less than n): 7

Encryption and Decryption
-----
Original Message : 7
Encrypted Message: 11
Decrypted Message: 7

```

RESULT:

Thus, the RSA algorithm was successfully implemented in C. The message 7 was encrypted to 224 using the public key and successfully decrypted back to 7 using the private key.